SKEIN ENGINE

VOLUME 12

BUILD, TESTING, DEPLOYMENT AND GOVERNANCE

Test architecture, CI, performance gates, security, packaging, documentation control, release policy and long-term technical governance.

DOCUMENT CODE	**SKEIN-IMP-002-V12**
VERSION	**0.1**
STATUS	**DRAFT CONSTRUCTION REFERENCE**
PLANNED PAGES	**150**
CHAPTERS	**14**
DATE	**30 July 2026**

IMPLEMENTATION MANUAL / PRINT SET

Current architecture source: SKEIN-SPEC-001 v0.4 and SKEIN-IMP-002-MASTER v0.1

12.01 Verification strategy

Purpose and boundary: test pyramid

This page defines the purpose and boundary for Verification strategy, with particular attention to test pyramid. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Test pyramid is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with evidence is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for test pyramid; distinguish required behavior from illustrative implementation detail.
- Keep evidence behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class VerificationStrategyService final
{
public:
    [[nodiscard]] Result<void> Initialise(const VerificationStrategyConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    VerificationStrategyState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.01 and test pyramid.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.01 Verification strategy

Architecture: evidence

This page defines the architecture for Verification strategy, with particular attention to evidence. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Evidence is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with ownership is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for evidence; distinguish required behavior from illustrative implementation detail.
- Keep ownership behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Verification", "VerificationStrategy");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.01 and evidence.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.01 Verification strategy

Data model: ownership

This page defines the data model for Verification strategy, with particular attention to ownership. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Ownership is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with failure triage is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for ownership; distinguish required behavior from illustrative implementation detail.
- Keep failure triage behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class VerificationStrategyService final
{
public:
    [[nodiscard]] Result<void> Initialise(const VerificationStrategyConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    VerificationStrategyState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.01 and ownership.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.01 Verification strategy

Public API: failure triage

This page defines the public api for Verification strategy, with particular attention to failure triage. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Failure triage is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with test pyramid is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for failure triage; distinguish required behavior from illustrative implementation detail.
- Keep test pyramid behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Verification", "VerificationStrategy");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.01 and failure triage.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.01 Verification strategy

Ownership and lifetime: test pyramid

This page defines the ownership and lifetime for Verification strategy, with particular attention to test pyramid. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Test pyramid is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with evidence is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for test pyramid; distinguish required behavior from illustrative implementation detail.
- Keep evidence behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class VerificationStrategyService final
{
public:
    [[nodiscard]] Result<void> Initialise(const VerificationStrategyConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    VerificationStrategyState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.01 and test pyramid.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.01 Verification strategy

Threading model: evidence

This page defines the threading model for Verification strategy, with particular attention to evidence. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Evidence is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with ownership is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for evidence; distinguish required behavior from illustrative implementation detail.
- Keep ownership behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Verification", "VerificationStrategy");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.01 and evidence.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.01 Verification strategy

Memory strategy: ownership

This page defines the memory strategy for Verification strategy, with particular attention to ownership. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Ownership is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with failure triage is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for ownership; distinguish required behavior from illustrative implementation detail.
- Keep failure triage behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class VerificationStrategyService final
{
public:
    [[nodiscard]] Result<void> Initialise(const VerificationStrategyConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    VerificationStrategyState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.01 and ownership.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.01 Verification strategy

Failure handling: failure triage

This page defines the failure handling for Verification strategy, with particular attention to failure triage. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Failure triage is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with test pyramid is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for failure triage; distinguish required behavior from illustrative implementation detail.
- Keep test pyramid behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Verification", "VerificationStrategy");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.01 and failure triage.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.01 Verification strategy

Diagnostics: test pyramid

This page defines the diagnostics for Verification strategy, with particular attention to test pyramid. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Test pyramid is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with evidence is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for test pyramid; distinguish required behavior from illustrative implementation detail.
- Keep evidence behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class VerificationStrategyService final
{
public:
    [[nodiscard]] Result<void> Initialise(const VerificationStrategyConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    VerificationStrategyState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.01 and test pyramid.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.02 Unit-test framework and conventions

Purpose and boundary: test discovery

This page defines the purpose and boundary for Unit-test framework and conventions, with particular attention to test discovery. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Test discovery is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with fixtures is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for test discovery; distinguish required behavior from illustrative implementation detail.
- Keep fixtures behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class UnitTestFrameworkService final
{
public:
    [[nodiscard]] Result<void> Initialise(const UnitTestFrameworkConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    UnitTestFrameworkState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.02 and test discovery.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.02 Unit-test framework and conventions

Architecture: fixtures

This page defines the architecture for Unit-test framework and conventions, with particular attention to fixtures. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Fixtures is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with assertions is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for fixtures; distinguish required behavior from illustrative implementation detail.
- Keep assertions behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Unit", "UnitTestFramework");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.02 and fixtures.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.02 Unit-test framework and conventions

Data model: assertions

This page defines the data model for Unit-test framework and conventions, with particular attention to assertions. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Assertions is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with determinism is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for assertions; distinguish required behavior from illustrative implementation detail.
- Keep determinism behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class UnitTestFrameworkService final
{
public:
    [[nodiscard]] Result<void> Initialise(const UnitTestFrameworkConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    UnitTestFrameworkState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.02 and assertions.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.02 Unit-test framework and conventions

Public API: determinism

This page defines the public api for Unit-test framework and conventions, with particular attention to determinism. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Determinism is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with timing is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for determinism; distinguish required behavior from illustrative implementation detail.
- Keep timing behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Unit", "UnitTestFramework");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.02 and determinism.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.02 Unit-test framework and conventions

Ownership and lifetime: timing

This page defines the ownership and lifetime for Unit-test framework and conventions, with particular attention to timing. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Timing is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with test discovery is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for timing; distinguish required behavior from illustrative implementation detail.
- Keep test discovery behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class UnitTestFrameworkService final
{
public:
    [[nodiscard]] Result<void> Initialise(const UnitTestFrameworkConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    UnitTestFrameworkState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.02 and timing.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.02 Unit-test framework and conventions

Threading model: test discovery

This page defines the threading model for Unit-test framework and conventions, with particular attention to test discovery. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Test discovery is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with fixtures is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for test discovery; distinguish required behavior from illustrative implementation detail.
- Keep fixtures behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Unit", "UnitTestFramework");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.02 and test discovery.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.02 Unit-test framework and conventions

Memory strategy: fixtures

This page defines the memory strategy for Unit-test framework and conventions, with particular attention to fixtures. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Fixtures is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with assertions is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for fixtures; distinguish required behavior from illustrative implementation detail.
- Keep assertions behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class UnitTestFrameworkService final
{
public:
    [[nodiscard]] Result<void> Initialise(const UnitTestFrameworkConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    UnitTestFrameworkState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.02 and fixtures.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.02 Unit-test framework and conventions

Failure handling: assertions

This page defines the failure handling for Unit-test framework and conventions, with particular attention to assertions. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Assertions is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with determinism is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for assertions; distinguish required behavior from illustrative implementation detail.
- Keep determinism behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Unit", "UnitTestFramework");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.02 and assertions.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.02 Unit-test framework and conventions

Diagnostics: determinism

This page defines the diagnostics for Unit-test framework and conventions, with particular attention to determinism. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Determinism is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with timing is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for determinism; distinguish required behavior from illustrative implementation detail.
- Keep timing behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class UnitTestFrameworkService final
{
public:
    [[nodiscard]] Result<void> Initialise(const UnitTestFrameworkConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    UnitTestFrameworkState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.02 and determinism.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.02 Unit-test framework and conventions

Implementation sequence: timing

This page defines the implementation sequence for Unit-test framework and conventions, with particular attention to timing. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Timing is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with test discovery is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for timing; distinguish required behavior from illustrative implementation detail.
- Keep test discovery behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Unit", "UnitTestFramework");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.02 and timing.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.02 Unit-test framework and conventions

Verification: test discovery

This page defines the verification for Unit-test framework and conventions, with particular attention to test discovery. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Test discovery is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with fixtures is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for test discovery; distinguish required behavior from illustrative implementation detail.
- Keep fixtures behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class UnitTestFrameworkService final
{
public:
    [[nodiscard]] Result<void> Initialise(const UnitTestFrameworkConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    UnitTestFrameworkState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.02 and test discovery.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.03 Integration and subsystem tests

Purpose and boundary: module harnesses

This page defines the purpose and boundary for Integration and subsystem tests, with particular attention to module harnesses. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Module harnesses is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with temporary projects is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for module harnesses; distinguish required behavior from illustrative implementation detail.
- Keep temporary projects behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class IntegrationAndSubsystemService final
{
public:
    [[nodiscard]] Result<void> Initialise(const IntegrationAndSubsystemConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    IntegrationAndSubsystemState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.03 and module harnesses.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.03 Integration and subsystem tests

Architecture: temporary projects

This page defines the architecture for Integration and subsystem tests, with particular attention to temporary projects. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Temporary projects is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with failure injection is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for temporary projects; distinguish required behavior from illustrative implementation detail.
- Keep failure injection behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Integration", "IntegrationAndSubsystem");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.03 and temporary projects.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.03 Integration and subsystem tests

Data model: failure injection

This page defines the data model for Integration and subsystem tests, with particular attention to failure injection. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Failure injection is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with cleanup is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for failure injection; distinguish required behavior from illustrative implementation detail.
- Keep cleanup behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class IntegrationAndSubsystemService final
{
public:
    [[nodiscard]] Result<void> Initialise(const IntegrationAndSubsystemConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    IntegrationAndSubsystemState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.03 and failure injection.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.03 Integration and subsystem tests

Public API: cleanup

This page defines the public api for Integration and subsystem tests, with particular attention to cleanup. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Cleanup is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with module harnesses is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for cleanup; distinguish required behavior from illustrative implementation detail.
- Keep module harnesses behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Integration", "IntegrationAndSubsystem");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.03 and cleanup.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.03 Integration and subsystem tests

Ownership and lifetime: module harnesses

This page defines the ownership and lifetime for Integration and subsystem tests, with particular attention to module harnesses. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Module harnesses is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with temporary projects is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for module harnesses; distinguish required behavior from illustrative implementation detail.
- Keep temporary projects behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class IntegrationAndSubsystemService final
{
public:
    [[nodiscard]] Result<void> Initialise(const IntegrationAndSubsystemConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    IntegrationAndSubsystemState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.03 and module harnesses.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.03 Integration and subsystem tests

Threading model: temporary projects

This page defines the threading model for Integration and subsystem tests, with particular attention to temporary projects. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Temporary projects is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with failure injection is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for temporary projects; distinguish required behavior from illustrative implementation detail.
- Keep failure injection behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Integration", "IntegrationAndSubsystem");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.03 and temporary projects.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.03 Integration and subsystem tests

Memory strategy: failure injection

This page defines the memory strategy for Integration and subsystem tests, with particular attention to failure injection. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Failure injection is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with cleanup is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for failure injection; distinguish required behavior from illustrative implementation detail.
- Keep cleanup behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class IntegrationAndSubsystemService final
{
public:
    [[nodiscard]] Result<void> Initialise(const IntegrationAndSubsystemConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    IntegrationAndSubsystemState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.03 and failure injection.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.03 Integration and subsystem tests

Failure handling: cleanup

This page defines the failure handling for Integration and subsystem tests, with particular attention to cleanup. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Cleanup is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with module harnesses is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for cleanup; distinguish required behavior from illustrative implementation detail.
- Keep module harnesses behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Integration", "IntegrationAndSubsystem");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.03 and cleanup.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.03 Integration and subsystem tests

Diagnostics: module harnesses

This page defines the diagnostics for Integration and subsystem tests, with particular attention to module harnesses. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Module harnesses is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with temporary projects is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for module harnesses; distinguish required behavior from illustrative implementation detail.
- Keep temporary projects behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class IntegrationAndSubsystemService final
{
public:
    [[nodiscard]] Result<void> Initialise(const IntegrationAndSubsystemConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    IntegrationAndSubsystemState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.03 and module harnesses.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.03 Integration and subsystem tests

Implementation sequence: temporary projects

This page defines the implementation sequence for Integration and subsystem tests, with particular attention to temporary projects. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Temporary projects is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with failure injection is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for temporary projects; distinguish required behavior from illustrative implementation detail.
- Keep failure injection behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Integration", "IntegrationAndSubsystem");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.03 and temporary projects.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.03 Integration and subsystem tests

Verification: failure injection

This page defines the verification for Integration and subsystem tests, with particular attention to failure injection. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Failure injection is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with cleanup is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for failure injection; distinguish required behavior from illustrative implementation detail.
- Keep cleanup behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class IntegrationAndSubsystemService final
{
public:
    [[nodiscard]] Result<void> Initialise(const IntegrationAndSubsystemConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    IntegrationAndSubsystemState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.03 and failure injection.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.03 Integration and subsystem tests

Completion gate: cleanup

This page defines the completion gate for Integration and subsystem tests, with particular attention to cleanup. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract lists the objective criteria that permit dependent chapters to begin.

Cleanup is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to enforce its inputs, isolate state transitions, and expose failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with module harnesses is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for cleanup; distinguish required behavior from illustrative implementation detail.
- Keep module harnesses behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Integration", "IntegrationAndSubsystem");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.03 and cleanup.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.03 Integration and subsystem tests

Purpose and boundary: module harnesses

This page defines the purpose and boundary for Integration and subsystem tests, with particular attention to module harnesses. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Module harnesses is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with temporary projects is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for module harnesses; distinguish required behavior from illustrative implementation detail.
- Keep temporary projects behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class IntegrationAndSubsystemService final
{
public:
    [[nodiscard]] Result<void> Initialise(const IntegrationAndSubsystemConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    IntegrationAndSubsystemState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.03 and module harnesses.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.04 Rendering image tests

Purpose and boundary: golden images

This page defines the purpose and boundary for Rendering image tests, with particular attention to golden images. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Golden images is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with tolerances is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for golden images; distinguish required behavior from illustrative implementation detail.
- Keep tolerances behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class RenderingImageTestsService final
{
public:
    [[nodiscard]] Result<void> Initialise(const RenderingImageTestsConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    RenderingImageTestsState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.04 and golden images.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.04 Rendering image tests

Architecture: tolerances

This page defines the architecture for Rendering image tests, with particular attention to tolerances. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Tolerances is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with backend matrix is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for tolerances; distinguish required behavior from illustrative implementation detail.
- Keep backend matrix behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Rendering", "RenderingImageTests");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.04 and tolerances.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.04 Rendering image tests

Data model: backend matrix

This page defines the data model for Rendering image tests, with particular attention to backend matrix. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Backend matrix is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with artifact review is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for backend matrix; distinguish required behavior from illustrative implementation detail.
- Keep artifact review behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class RenderingImageTestsService final
{
public:
    [[nodiscard]] Result<void> Initialise(const RenderingImageTestsConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    RenderingImageTestsState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.04 and backend matrix.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.04 Rendering image tests

Public API: artifact review

This page defines the public api for Rendering image tests, with particular attention to artifact review. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Artifact review is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with golden images is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for artifact review; distinguish required behavior from illustrative implementation detail.
- Keep golden images behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Rendering", "RenderingImageTests");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.04 and artifact review.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.04 Rendering image tests

Ownership and lifetime: golden images

This page defines the ownership and lifetime for Rendering image tests, with particular attention to golden images. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Golden images is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with tolerances is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for golden images; distinguish required behavior from illustrative implementation detail.
- Keep tolerances behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class RenderingImageTestsService final
{
public:
    [[nodiscard]] Result<void> Initialise(const RenderingImageTestsConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    RenderingImageTestsState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.04 and golden images.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.04 Rendering image tests

Threading model: tolerances

This page defines the threading model for Rendering image tests, with particular attention to tolerances. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Tolerances is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with backend matrix is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for tolerances; distinguish required behavior from illustrative implementation detail.
- Keep backend matrix behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Rendering", "RenderingImageTests");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.04 and tolerances.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.04 Rendering image tests

Memory strategy: backend matrix

This page defines the memory strategy for Rendering image tests, with particular attention to backend matrix. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Backend matrix is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with artifact review is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for backend matrix; distinguish required behavior from illustrative implementation detail.
- Keep artifact review behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class RenderingImageTestsService final
{
public:
    [[nodiscard]] Result<void> Initialise(const RenderingImageTestsConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    RenderingImageTestsState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.04 and backend matrix.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.04 Rendering image tests

Failure handling: artifact review

This page defines the failure handling for Rendering image tests, with particular attention to artifact review. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Artifact review is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with golden images is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for artifact review; distinguish required behavior from illustrative implementation detail.
- Keep golden images behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Rendering", "RenderingImageTests");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.04 and artifact review.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.04 Rendering image tests

Diagnostics: golden images

This page defines the diagnostics for Rendering image tests, with particular attention to golden images. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Golden images is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with tolerances is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for golden images; distinguish required behavior from illustrative implementation detail.
- Keep tolerances behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class RenderingImageTestsService final
{
public:
    [[nodiscard]] Result<void> Initialise(const RenderingImageTestsConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    RenderingImageTestsState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.04 and golden images.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.04 Rendering image tests

Implementation sequence: tolerances

This page defines the implementation sequence for Rendering image tests, with particular attention to tolerances. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Tolerances is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with backend matrix is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for tolerances; distinguish required behavior from illustrative implementation detail.
- Keep backend matrix behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Rendering", "RenderingImageTests");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.04 and tolerances.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.04 Rendering image tests

Verification: backend matrix

This page defines the verification for Rendering image tests, with particular attention to backend matrix. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Backend matrix is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with artifact review is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for backend matrix; distinguish required behavior from illustrative implementation detail.
- Keep artifact review behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class RenderingImageTestsService final
{
public:
    [[nodiscard]] Result<void> Initialise(const RenderingImageTestsConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    RenderingImageTestsState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.04 and backend matrix.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.05 Performance benchmark programme

Purpose and boundary: microbenchmarks

This page defines the purpose and boundary for Performance benchmark programme, with particular attention to microbenchmarks. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Microbenchmarks is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with scene benchmarks is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for microbenchmarks; distinguish required behavior from illustrative implementation detail.
- Keep scene benchmarks behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class PerformanceBenchmarkProgrammeService final
{
public:
    [[nodiscard]] Result<void> Initialise(const PerformanceBenchmarkProgrammeConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    PerformanceBenchmarkProgrammeState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.05 and microbenchmarks.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.05 Performance benchmark programme

Architecture: scene benchmarks

This page defines the architecture for Performance benchmark programme, with particular attention to scene benchmarks. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Scene benchmarks is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with budgets is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for scene benchmarks; distinguish required behavior from illustrative implementation detail.
- Keep budgets behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Performance", "PerformanceBenchmarkProgramme");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.05 and scene benchmarks.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.05 Performance benchmark programme

Data model: budgets

This page defines the data model for Performance benchmark programme, with particular attention to budgets. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Budgets is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with regression thresholds is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for budgets; distinguish required behavior from illustrative implementation detail.
- Keep regression thresholds behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class PerformanceBenchmarkProgrammeService final
{
public:
    [[nodiscard]] Result<void> Initialise(const PerformanceBenchmarkProgrammeConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    PerformanceBenchmarkProgrammeState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.05 and budgets.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.05 Performance benchmark programme

Public API: regression thresholds

This page defines the public api for Performance benchmark programme, with particular attention to regression thresholds. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Regression thresholds is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with microbenchmarks is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for regression thresholds; distinguish required behavior from illustrative implementation detail.
- Keep microbenchmarks behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Performance", "PerformanceBenchmarkProgramme");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.05 and regression thresholds.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.05 Performance benchmark programme

Ownership and lifetime: microbenchmarks

This page defines the ownership and lifetime for Performance benchmark programme, with particular attention to microbenchmarks. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Microbenchmarks is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with scene benchmarks is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for microbenchmarks; distinguish required behavior from illustrative implementation detail.
- Keep scene benchmarks behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class PerformanceBenchmarkProgrammeService final
{
public:
    [[nodiscard]] Result<void> Initialise(const PerformanceBenchmarkProgrammeConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    PerformanceBenchmarkProgrammeState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.05 and microbenchmarks.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.05 Performance benchmark programme

Threading model: scene benchmarks

This page defines the threading model for Performance benchmark programme, with particular attention to scene benchmarks. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Scene benchmarks is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with budgets is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for scene benchmarks; distinguish required behavior from illustrative implementation detail.
- Keep budgets behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Performance", "PerformanceBenchmarkProgramme");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.05 and scene benchmarks.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.05 Performance benchmark programme

Memory strategy: budgets

This page defines the memory strategy for Performance benchmark programme, with particular attention to budgets. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Budgets is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with regression thresholds is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for budgets; distinguish required behavior from illustrative implementation detail.
- Keep regression thresholds behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class PerformanceBenchmarkProgrammeService final
{
public:
    [[nodiscard]] Result<void> Initialise(const PerformanceBenchmarkProgrammeConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    PerformanceBenchmarkProgrammeState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.05 and budgets.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.05 Performance benchmark programme

Failure handling: regression thresholds

This page defines the failure handling for Performance benchmark programme, with particular attention to regression thresholds. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Regression thresholds is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with microbenchmarks is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for regression thresholds; distinguish required behavior from illustrative implementation detail.
- Keep microbenchmarks behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Performance", "PerformanceBenchmarkProgramme");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.05 and regression thresholds.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.05 Performance benchmark programme

Diagnostics: microbenchmarks

This page defines the diagnostics for Performance benchmark programme, with particular attention to microbenchmarks. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Microbenchmarks is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with scene benchmarks is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for microbenchmarks; distinguish required behavior from illustrative implementation detail.
- Keep scene benchmarks behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class PerformanceBenchmarkProgrammeService final
{
public:
    [[nodiscard]] Result<void> Initialise(const PerformanceBenchmarkProgrammeConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    PerformanceBenchmarkProgrammeState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.05 and microbenchmarks.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.05 Performance benchmark programme

Implementation sequence: scene benchmarks

This page defines the implementation sequence for Performance benchmark programme, with particular attention to scene benchmarks. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Scene benchmarks is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with budgets is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for scene benchmarks; distinguish required behavior from illustrative implementation detail.
- Keep budgets behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Performance", "PerformanceBenchmarkProgramme");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.05 and scene benchmarks.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.05 Performance benchmark programme

Verification: budgets

This page defines the verification for Performance benchmark programme, with particular attention to budgets. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Budgets is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with regression thresholds is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for budgets; distinguish required behavior from illustrative implementation detail.
- Keep regression thresholds behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class PerformanceBenchmarkProgrammeService final
{
public:
    [[nodiscard]] Result<void> Initialise(const PerformanceBenchmarkProgrammeConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    PerformanceBenchmarkProgrammeState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.05 and budgets.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.05 Performance benchmark programme

Completion gate: regression thresholds

This page defines the completion gate for Performance benchmark programme, with particular attention to regression thresholds. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract lists the objective criteria that permit dependent chapters to begin.

Regression thresholds is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to enforce its inputs, isolate state transitions, and expose failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with microbenchmarks is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for regression thresholds; distinguish required behavior from illustrative implementation detail.
- Keep microbenchmarks behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Performance", "PerformanceBenchmarkProgramme");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.05 and regression thresholds.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.05 Performance benchmark programme

Purpose and boundary: microbenchmarks

This page defines the purpose and boundary for Performance benchmark programme, with particular attention to microbenchmarks. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Microbenchmarks is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with scene benchmarks is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for microbenchmarks; distinguish required behavior from illustrative implementation detail.
- Keep scene benchmarks behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class PerformanceBenchmarkProgrammeService final
{
public:
    [[nodiscard]] Result<void> Initialise(const PerformanceBenchmarkProgrammeConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    PerformanceBenchmarkProgrammeState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.05 and microbenchmarks.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.06 Memory and long-running stability tests

Purpose and boundary: budgets

This page defines the purpose and boundary for Memory and long-running stability tests, with particular attention to budgets. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Budgets is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with leaks is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for budgets; distinguish required behavior from illustrative implementation detail.
- Keep leaks behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class MemoryAndLongService final
{
public:
    [[nodiscard]] Result<void> Initialise(const MemoryAndLongConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    MemoryAndLongState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.06 and budgets.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.06 Memory and long-running stability tests

Architecture: leaks

This page defines the architecture for Memory and long-running stability tests, with particular attention to leaks. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Leaks is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with fragmentation is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for leaks; distinguish required behavior from illustrative implementation detail.
- Keep fragmentation behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Memory", "MemoryAndLong");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.06 and leaks.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.06 Memory and long-running stability tests

Data model: fragmentation

This page defines the data model for Memory and long-running stability tests, with particular attention to fragmentation. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Fragmentation is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with soak tests is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for fragmentation; distinguish required behavior from illustrative implementation detail.
- Keep soak tests behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class MemoryAndLongService final
{
public:
    [[nodiscard]] Result<void> Initialise(const MemoryAndLongConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    MemoryAndLongState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.06 and fragmentation.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.06 Memory and long-running stability tests

Public API: soak tests

This page defines the public api for Memory and long-running stability tests, with particular attention to soak tests. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Soak tests is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with shutdown is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for soak tests; distinguish required behavior from illustrative implementation detail.
- Keep shutdown behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Memory", "MemoryAndLong");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.06 and soak tests.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.06 Memory and long-running stability tests

Ownership and lifetime: shutdown

This page defines the ownership and lifetime for Memory and long-running stability tests, with particular attention to shutdown. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Shutdown is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with budgets is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for shutdown; distinguish required behavior from illustrative implementation detail.
- Keep budgets behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class MemoryAndLongService final
{
public:
    [[nodiscard]] Result<void> Initialise(const MemoryAndLongConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    MemoryAndLongState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.06 and shutdown.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.06 Memory and long-running stability tests

Threading model: budgets

This page defines the threading model for Memory and long-running stability tests, with particular attention to budgets. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Budgets is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with leaks is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for budgets; distinguish required behavior from illustrative implementation detail.
- Keep leaks behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Memory", "MemoryAndLong");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.06 and budgets.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.06 Memory and long-running stability tests

Memory strategy: leaks

This page defines the memory strategy for Memory and long-running stability tests, with particular attention to leaks. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Leaks is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with fragmentation is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for leaks; distinguish required behavior from illustrative implementation detail.
- Keep fragmentation behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class MemoryAndLongService final
{
public:
    [[nodiscard]] Result<void> Initialise(const MemoryAndLongConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    MemoryAndLongState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.06 and leaks.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.06 Memory and long-running stability tests

Failure handling: fragmentation

This page defines the failure handling for Memory and long-running stability tests, with particular attention to fragmentation. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Fragmentation is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with soak tests is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for fragmentation; distinguish required behavior from illustrative implementation detail.
- Keep soak tests behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Memory", "MemoryAndLong");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.06 and fragmentation.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.06 Memory and long-running stability tests

Diagnostics: soak tests

This page defines the diagnostics for Memory and long-running stability tests, with particular attention to soak tests. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Soak tests is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with shutdown is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for soak tests; distinguish required behavior from illustrative implementation detail.
- Keep shutdown behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class MemoryAndLongService final
{
public:
    [[nodiscard]] Result<void> Initialise(const MemoryAndLongConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    MemoryAndLongState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.06 and soak tests.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.06 Memory and long-running stability tests

Implementation sequence: shutdown

This page defines the implementation sequence for Memory and long-running stability tests, with particular attention to shutdown. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Shutdown is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with budgets is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for shutdown; distinguish required behavior from illustrative implementation detail.
- Keep budgets behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Memory", "MemoryAndLong");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.06 and shutdown.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.06 Memory and long-running stability tests

Verification: budgets

This page defines the verification for Memory and long-running stability tests, with particular attention to budgets. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Budgets is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with leaks is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for budgets; distinguish required behavior from illustrative implementation detail.
- Keep leaks behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class MemoryAndLongService final
{
public:
    [[nodiscard]] Result<void> Initialise(const MemoryAndLongConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    MemoryAndLongState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.06 and budgets.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.07 Continuous integration matrix

Purpose and boundary: four presets

This page defines the purpose and boundary for Continuous integration matrix, with particular attention to four presets. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Four presets is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with clean builds is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for four presets; distinguish required behavior from illustrative implementation detail.
- Keep clean builds behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ContinuousIntegrationMatrixService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ContinuousIntegrationMatrixConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ContinuousIntegrationMatrixState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.07 and four presets.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.07 Continuous integration matrix

Architecture: clean builds

This page defines the architecture for Continuous integration matrix, with particular attention to clean builds. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Clean builds is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with tests is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for clean builds; distinguish required behavior from illustrative implementation detail.
- Keep tests behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Continuous", "ContinuousIntegrationMatrix");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.07 and clean builds.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.07 Continuous integration matrix

Data model: tests

This page defines the data model for Continuous integration matrix, with particular attention to tests. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Tests is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with artifacts is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for tests; distinguish required behavior from illustrative implementation detail.
- Keep artifacts behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ContinuousIntegrationMatrixService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ContinuousIntegrationMatrixConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ContinuousIntegrationMatrixState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.07 and tests.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.07 Continuous integration matrix

Public API: artifacts

This page defines the public api for Continuous integration matrix, with particular attention to artifacts. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Artifacts is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with caches is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for artifacts; distinguish required behavior from illustrative implementation detail.
- Keep caches behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Continuous", "ContinuousIntegrationMatrix");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.07 and artifacts.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.07 Continuous integration matrix

Ownership and lifetime: caches

This page defines the ownership and lifetime for Continuous integration matrix, with particular attention to caches. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Caches is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with four presets is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for caches; distinguish required behavior from illustrative implementation detail.
- Keep four presets behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ContinuousIntegrationMatrixService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ContinuousIntegrationMatrixConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ContinuousIntegrationMatrixState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.07 and caches.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.07 Continuous integration matrix

Threading model: four presets

This page defines the threading model for Continuous integration matrix, with particular attention to four presets. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Four presets is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with clean builds is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for four presets; distinguish required behavior from illustrative implementation detail.
- Keep clean builds behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Continuous", "ContinuousIntegrationMatrix");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.07 and four presets.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.07 Continuous integration matrix

Memory strategy: clean builds

This page defines the memory strategy for Continuous integration matrix, with particular attention to clean builds. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Clean builds is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with tests is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for clean builds; distinguish required behavior from illustrative implementation detail.
- Keep tests behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ContinuousIntegrationMatrixService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ContinuousIntegrationMatrixConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ContinuousIntegrationMatrixState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.07 and clean builds.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.07 Continuous integration matrix

Failure handling: tests

This page defines the failure handling for Continuous integration matrix, with particular attention to tests. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Tests is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with artifacts is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for tests; distinguish required behavior from illustrative implementation detail.
- Keep artifacts behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Continuous", "ContinuousIntegrationMatrix");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.07 and tests.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.07 Continuous integration matrix

Diagnostics: artifacts

This page defines the diagnostics for Continuous integration matrix, with particular attention to artifacts. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Artifacts is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with caches is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for artifacts; distinguish required behavior from illustrative implementation detail.
- Keep caches behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ContinuousIntegrationMatrixService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ContinuousIntegrationMatrixConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ContinuousIntegrationMatrixState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.07 and artifacts.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.07 Continuous integration matrix

Implementation sequence: caches

This page defines the implementation sequence for Continuous integration matrix, with particular attention to caches. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Caches is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with four presets is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for caches; distinguish required behavior from illustrative implementation detail.
- Keep four presets behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Continuous", "ContinuousIntegrationMatrix");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.07 and caches.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.07 Continuous integration matrix

Verification: four presets

This page defines the verification for Continuous integration matrix, with particular attention to four presets. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Four presets is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with clean builds is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for four presets; distinguish required behavior from illustrative implementation detail.
- Keep clean builds behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ContinuousIntegrationMatrixService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ContinuousIntegrationMatrixConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ContinuousIntegrationMatrixState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.07 and four presets.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.07 Continuous integration matrix

Completion gate: clean builds

This page defines the completion gate for Continuous integration matrix, with particular attention to clean builds. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract lists the objective criteria that permit dependent chapters to begin.

Clean builds is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to enforce its inputs, isolate state transitions, and expose failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with tests is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for clean builds; distinguish required behavior from illustrative implementation detail.
- Keep tests behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Continuous", "ContinuousIntegrationMatrix");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.07 and clean builds.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.08 Static analysis and sanitizers

Purpose and boundary: clang-tidy

This page defines the purpose and boundary for Static analysis and sanitizers, with particular attention to clang-tidy. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Clang-tidy is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with MSVC analysis is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for clang-tidy; distinguish required behavior from illustrative implementation detail.
- Keep MSVC analysis behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class StaticAnalysisAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const StaticAnalysisAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    StaticAnalysisAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.08 and clang-tidy.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.08 Static analysis and sanitizers

Architecture: MSVC analysis

This page defines the architecture for Static analysis and sanitizers, with particular attention to MSVC analysis. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Msvc analysis is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with ASan where viable is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for MSVC analysis; distinguish required behavior from illustrative implementation detail.
- Keep ASan where viable behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Static", "StaticAnalysisAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.08 and MSVC analysis.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.08 Static analysis and sanitizers

Data model: ASan where viable

This page defines the data model for Static analysis and sanitizers, with particular attention to ASan where viable. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Asan where viable is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with warnings is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for ASan where viable; distinguish required behavior from illustrative implementation detail.
- Keep warnings behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class StaticAnalysisAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const StaticAnalysisAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    StaticAnalysisAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.08 and ASan where viable.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.08 Static analysis and sanitizers

Public API: warnings

This page defines the public api for Static analysis and sanitizers, with particular attention to warnings. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Warnings is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with clang-tidy is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for warnings; distinguish required behavior from illustrative implementation detail.
- Keep clang-tidy behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Static", "StaticAnalysisAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.08 and warnings.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.08 Static analysis and sanitizers

Ownership and lifetime: clang-tidy

This page defines the ownership and lifetime for Static analysis and sanitizers, with particular attention to clang-tidy. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Clang-tidy is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with MSVC analysis is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for clang-tidy; distinguish required behavior from illustrative implementation detail.
- Keep MSVC analysis behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class StaticAnalysisAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const StaticAnalysisAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    StaticAnalysisAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.08 and clang-tidy.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.08 Static analysis and sanitizers

Threading model: MSVC analysis

This page defines the threading model for Static analysis and sanitizers, with particular attention to MSVC analysis. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Msvc analysis is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with ASan where viable is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for MSVC analysis; distinguish required behavior from illustrative implementation detail.
- Keep ASan where viable behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Static", "StaticAnalysisAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.08 and MSVC analysis.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.08 Static analysis and sanitizers

Memory strategy: ASan where viable

This page defines the memory strategy for Static analysis and sanitizers, with particular attention to ASan where viable. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Asan where viable is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with warnings is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for ASan where viable; distinguish required behavior from illustrative implementation detail.
- Keep warnings behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class StaticAnalysisAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const StaticAnalysisAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    StaticAnalysisAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.08 and ASan where viable.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.08 Static analysis and sanitizers

Failure handling: warnings

This page defines the failure handling for Static analysis and sanitizers, with particular attention to warnings. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Warnings is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with clang-tidy is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for warnings; distinguish required behavior from illustrative implementation detail.
- Keep clang-tidy behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Static", "StaticAnalysisAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.08 and warnings.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.08 Static analysis and sanitizers

Diagnostics: clang-tidy

This page defines the diagnostics for Static analysis and sanitizers, with particular attention to clang-tidy. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Clang-tidy is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with MSVC analysis is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for clang-tidy; distinguish required behavior from illustrative implementation detail.
- Keep MSVC analysis behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class StaticAnalysisAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const StaticAnalysisAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    StaticAnalysisAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.08 and clang-tidy.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.09 Security and robustness

Purpose and boundary: input limits

This page defines the purpose and boundary for Security and robustness, with particular attention to input limits. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Input limits is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with fuzzing is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for input limits; distinguish required behavior from illustrative implementation detail.
- Keep fuzzing behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SecurityAndRobustnessService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SecurityAndRobustnessConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SecurityAndRobustnessState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.09 and input limits.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.09 Security and robustness

Architecture: fuzzing

This page defines the architecture for Security and robustness, with particular attention to fuzzing. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Fuzzing is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with untrusted assets is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for fuzzing; distinguish required behavior from illustrative implementation detail.
- Keep untrusted assets behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Security", "SecurityAndRobustness");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.09 and fuzzing.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.09 Security and robustness

Data model: untrusted assets

This page defines the data model for Security and robustness, with particular attention to untrusted assets. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Untrusted assets is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with plugins is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for untrusted assets; distinguish required behavior from illustrative implementation detail.
- Keep plugins behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SecurityAndRobustnessService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SecurityAndRobustnessConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SecurityAndRobustnessState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.09 and untrusted assets.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.09 Security and robustness

Public API: plugins

This page defines the public api for Security and robustness, with particular attention to plugins. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Plugins is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with network is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for plugins; distinguish required behavior from illustrative implementation detail.
- Keep network behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Security", "SecurityAndRobustness");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.09 and plugins.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.09 Security and robustness

Ownership and lifetime: network

This page defines the ownership and lifetime for Security and robustness, with particular attention to network. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Network is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with input limits is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for network; distinguish required behavior from illustrative implementation detail.
- Keep input limits behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SecurityAndRobustnessService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SecurityAndRobustnessConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SecurityAndRobustnessState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.09 and network.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.09 Security and robustness

Threading model: input limits

This page defines the threading model for Security and robustness, with particular attention to input limits. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Input limits is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with fuzzing is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for input limits; distinguish required behavior from illustrative implementation detail.
- Keep fuzzing behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Security", "SecurityAndRobustness");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.09 and input limits.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.09 Security and robustness

Memory strategy: fuzzing

This page defines the memory strategy for Security and robustness, with particular attention to fuzzing. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Fuzzing is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with untrusted assets is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for fuzzing; distinguish required behavior from illustrative implementation detail.
- Keep untrusted assets behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SecurityAndRobustnessService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SecurityAndRobustnessConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SecurityAndRobustnessState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.09 and fuzzing.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.09 Security and robustness

Failure handling: untrusted assets

This page defines the failure handling for Security and robustness, with particular attention to untrusted assets. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Untrusted assets is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with plugins is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for untrusted assets; distinguish required behavior from illustrative implementation detail.
- Keep plugins behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Security", "SecurityAndRobustness");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.09 and untrusted assets.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.09 Security and robustness

Diagnostics: plugins

This page defines the diagnostics for Security and robustness, with particular attention to plugins. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Plugins is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with network is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for plugins; distinguish required behavior from illustrative implementation detail.
- Keep network behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SecurityAndRobustnessService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SecurityAndRobustnessConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SecurityAndRobustnessState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.09 and plugins.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.09 Security and robustness

Implementation sequence: network

This page defines the implementation sequence for Security and robustness, with particular attention to network. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Network is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with input limits is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for network; distinguish required behavior from illustrative implementation detail.
- Keep input limits behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Security", "SecurityAndRobustness");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.09 and network.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.09 Security and robustness

Verification: input limits

This page defines the verification for Security and robustness, with particular attention to input limits. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Input limits is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with fuzzing is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for input limits; distinguish required behavior from illustrative implementation detail.
- Keep fuzzing behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SecurityAndRobustnessService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SecurityAndRobustnessConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SecurityAndRobustnessState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.09 and input limits.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.10 Cooking, packaging and deployment

Purpose and boundary: client

This page defines the purpose and boundary for Cooking, packaging and deployment, with particular attention to client. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Client is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with editor is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for client; distinguish required behavior from illustrative implementation detail.
- Keep editor behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class CookingPackagingAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const CookingPackagingAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    CookingPackagingAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.10 and client.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.10 Cooking, packaging and deployment

Architecture: editor

This page defines the architecture for Cooking, packaging and deployment, with particular attention to editor. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Editor is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with server is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for editor; distinguish required behavior from illustrative implementation detail.
- Keep server behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Cooking", "CookingPackagingAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.10 and editor.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.10 Cooking, packaging and deployment

Data model: server

This page defines the data model for Cooking, packaging and deployment, with particular attention to server. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Server is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with symbols is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for server; distinguish required behavior from illustrative implementation detail.
- Keep symbols behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class CookingPackagingAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const CookingPackagingAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    CookingPackagingAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.10 and server.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.10 Cooking, packaging and deployment

Public API: symbols

This page defines the public api for Cooking, packaging and deployment, with particular attention to symbols. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Symbols is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with manifests is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for symbols; distinguish required behavior from illustrative implementation detail.
- Keep manifests behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Cooking", "CookingPackagingAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.10 and symbols.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.10 Cooking, packaging and deployment

Ownership and lifetime: manifests

This page defines the ownership and lifetime for Cooking, packaging and deployment, with particular attention to manifests. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Manifests is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with install layouts is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for manifests; distinguish required behavior from illustrative implementation detail.
- Keep install layouts behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class CookingPackagingAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const CookingPackagingAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    CookingPackagingAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.10 and manifests.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.10 Cooking, packaging and deployment

Threading model: install layouts

This page defines the threading model for Cooking, packaging and deployment, with particular attention to install layouts. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Install layouts is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with client is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for install layouts; distinguish required behavior from illustrative implementation detail.
- Keep client behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Cooking", "CookingPackagingAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.10 and install layouts.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.10 Cooking, packaging and deployment

Memory strategy: client

This page defines the memory strategy for Cooking, packaging and deployment, with particular attention to client. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Client is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with editor is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for client; distinguish required behavior from illustrative implementation detail.
- Keep editor behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class CookingPackagingAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const CookingPackagingAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    CookingPackagingAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.10 and client.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.10 Cooking, packaging and deployment

Failure handling: editor

This page defines the failure handling for Cooking, packaging and deployment, with particular attention to editor. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Editor is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with server is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for editor; distinguish required behavior from illustrative implementation detail.
- Keep server behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Cooking", "CookingPackagingAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.10 and editor.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.10 Cooking, packaging and deployment

Diagnostics: server

This page defines the diagnostics for Cooking, packaging and deployment, with particular attention to server. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Server is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with symbols is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for server; distinguish required behavior from illustrative implementation detail.
- Keep symbols behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class CookingPackagingAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const CookingPackagingAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    CookingPackagingAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.10 and server.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.10 Cooking, packaging and deployment

Implementation sequence: symbols

This page defines the implementation sequence for Cooking, packaging and deployment, with particular attention to symbols. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Symbols is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with manifests is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for symbols; distinguish required behavior from illustrative implementation detail.
- Keep manifests behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Cooking", "CookingPackagingAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.10 and symbols.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.10 Cooking, packaging and deployment

Verification: manifests

This page defines the verification for Cooking, packaging and deployment, with particular attention to manifests. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Manifests is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with install layouts is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for manifests; distinguish required behavior from illustrative implementation detail.
- Keep install layouts behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class CookingPackagingAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const CookingPackagingAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    CookingPackagingAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.10 and manifests.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.11 Versioning and compatibility policy

Purpose and boundary: engine versions

This page defines the purpose and boundary for Versioning and compatibility policy, with particular attention to engine versions. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Engine versions is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with plugin API is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for engine versions; distinguish required behavior from illustrative implementation detail.
- Keep plugin API behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class VersioningAndCompatibilityService final
{
public:
    [[nodiscard]] Result<void> Initialise(const VersioningAndCompatibilityConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    VersioningAndCompatibilityState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.11 and engine versions.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.11 Versioning and compatibility policy

Architecture: plugin API

This page defines the architecture for Versioning and compatibility policy, with particular attention to plugin API. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Plugin api is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with assets is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for plugin API; distinguish required behavior from illustrative implementation detail.
- Keep assets behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Versioning", "VersioningAndCompatibility");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.11 and plugin API.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.11 Versioning and compatibility policy

Data model: assets

This page defines the data model for Versioning and compatibility policy, with particular attention to assets. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Assets is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with scripts is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for assets; distinguish required behavior from illustrative implementation detail.
- Keep scripts behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class VersioningAndCompatibilityService final
{
public:
    [[nodiscard]] Result<void> Initialise(const VersioningAndCompatibilityConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    VersioningAndCompatibilityState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.11 and assets.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.11 Versioning and compatibility policy

Public API: scripts

This page defines the public api for Versioning and compatibility policy, with particular attention to scripts. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Scripts is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with network protocols is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for scripts; distinguish required behavior from illustrative implementation detail.
- Keep network protocols behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Versioning", "VersioningAndCompatibility");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.11 and scripts.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.11 Versioning and compatibility policy

Ownership and lifetime: network protocols

This page defines the ownership and lifetime for Versioning and compatibility policy, with particular attention to network protocols. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Network protocols is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with engine versions is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for network protocols; distinguish required behavior from illustrative implementation detail.
- Keep engine versions behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class VersioningAndCompatibilityService final
{
public:
    [[nodiscard]] Result<void> Initialise(const VersioningAndCompatibilityConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    VersioningAndCompatibilityState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.11 and network protocols.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.11 Versioning and compatibility policy

Threading model: engine versions

This page defines the threading model for Versioning and compatibility policy, with particular attention to engine versions. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Engine versions is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with plugin API is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for engine versions; distinguish required behavior from illustrative implementation detail.
- Keep plugin API behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Versioning", "VersioningAndCompatibility");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.11 and engine versions.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.11 Versioning and compatibility policy

Memory strategy: plugin API

This page defines the memory strategy for Versioning and compatibility policy, with particular attention to plugin API. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Plugin api is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with assets is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for plugin API; distinguish required behavior from illustrative implementation detail.
- Keep assets behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class VersioningAndCompatibilityService final
{
public:
    [[nodiscard]] Result<void> Initialise(const VersioningAndCompatibilityConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    VersioningAndCompatibilityState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.11 and plugin API.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.11 Versioning and compatibility policy

Failure handling: assets

This page defines the failure handling for Versioning and compatibility policy, with particular attention to assets. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Assets is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with scripts is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for assets; distinguish required behavior from illustrative implementation detail.
- Keep scripts behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Versioning", "VersioningAndCompatibility");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.11 and assets.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.11 Versioning and compatibility policy

Diagnostics: scripts

This page defines the diagnostics for Versioning and compatibility policy, with particular attention to scripts. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Scripts is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with network protocols is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for scripts; distinguish required behavior from illustrative implementation detail.
- Keep network protocols behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class VersioningAndCompatibilityService final
{
public:
    [[nodiscard]] Result<void> Initialise(const VersioningAndCompatibilityConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    VersioningAndCompatibilityState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.11 and scripts.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.12 Documentation system

Purpose and boundary: master spec

This page defines the purpose and boundary for Documentation system, with particular attention to master spec. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Master spec is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with construction volumes is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for master spec; distinguish required behavior from illustrative implementation detail.
- Keep construction volumes behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class DocumentationSystemService final
{
public:
    [[nodiscard]] Result<void> Initialise(const DocumentationSystemConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    DocumentationSystemState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.12 and master spec.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.12 Documentation system

Architecture: construction volumes

This page defines the architecture for Documentation system, with particular attention to construction volumes. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Construction volumes is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with ADRs is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for construction volumes; distinguish required behavior from illustrative implementation detail.
- Keep ADRs behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Documentation", "DocumentationSystem");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.12 and construction volumes.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.12 Documentation system

Data model: ADRs

This page defines the data model for Documentation system, with particular attention to ADRs. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Adrs is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with API docs is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for ADRs; distinguish required behavior from illustrative implementation detail.
- Keep API docs behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class DocumentationSystemService final
{
public:
    [[nodiscard]] Result<void> Initialise(const DocumentationSystemConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    DocumentationSystemState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.12 and ADRs.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.12 Documentation system

Public API: API docs

This page defines the public api for Documentation system, with particular attention to API docs. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Api docs is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with changelogs is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for API docs; distinguish required behavior from illustrative implementation detail.
- Keep changelogs behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Documentation", "DocumentationSystem");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.12 and API docs.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.12 Documentation system

Ownership and lifetime: changelogs

This page defines the ownership and lifetime for Documentation system, with particular attention to changelogs. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Changelogs is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with master spec is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for changelogs; distinguish required behavior from illustrative implementation detail.
- Keep master spec behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class DocumentationSystemService final
{
public:
    [[nodiscard]] Result<void> Initialise(const DocumentationSystemConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    DocumentationSystemState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.12 and changelogs.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.12 Documentation system

Threading model: master spec

This page defines the threading model for Documentation system, with particular attention to master spec. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Master spec is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with construction volumes is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for master spec; distinguish required behavior from illustrative implementation detail.
- Keep construction volumes behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Documentation", "DocumentationSystem");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.12 and master spec.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.12 Documentation system

Memory strategy: construction volumes

This page defines the memory strategy for Documentation system, with particular attention to construction volumes. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Construction volumes is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with ADRs is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for construction volumes; distinguish required behavior from illustrative implementation detail.
- Keep ADRs behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class DocumentationSystemService final
{
public:
    [[nodiscard]] Result<void> Initialise(const DocumentationSystemConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    DocumentationSystemState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.12 and construction volumes.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.12 Documentation system

Failure handling: ADRs

This page defines the failure handling for Documentation system, with particular attention to ADRs. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Adrs is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with API docs is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for ADRs; distinguish required behavior from illustrative implementation detail.
- Keep API docs behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Documentation", "DocumentationSystem");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.12 and ADRs.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.12 Documentation system

Diagnostics: API docs

This page defines the diagnostics for Documentation system, with particular attention to API docs. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Api docs is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with changelogs is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for API docs; distinguish required behavior from illustrative implementation detail.
- Keep changelogs behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class DocumentationSystemService final
{
public:
    [[nodiscard]] Result<void> Initialise(const DocumentationSystemConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    DocumentationSystemState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.12 and API docs.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.13 Change control and governance

Purpose and boundary: scope changes

This page defines the purpose and boundary for Change control and governance, with particular attention to scope changes. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Scope changes is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with review gates is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for scope changes; distinguish required behavior from illustrative implementation detail.
- Keep review gates behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ChangeControlAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ChangeControlAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ChangeControlAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.13 and scope changes.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.13 Change control and governance

Architecture: review gates

This page defines the architecture for Change control and governance, with particular attention to review gates. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Review gates is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with deprecations is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for review gates; distinguish required behavior from illustrative implementation detail.
- Keep deprecations behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Change", "ChangeControlAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.13 and review gates.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.13 Change control and governance

Data model: deprecations

This page defines the data model for Change control and governance, with particular attention to deprecations. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Deprecations is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with decision records is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for deprecations; distinguish required behavior from illustrative implementation detail.
- Keep decision records behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ChangeControlAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ChangeControlAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ChangeControlAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.13 and deprecations.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.13 Change control and governance

Public API: decision records

This page defines the public api for Change control and governance, with particular attention to decision records. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Decision records is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with scope changes is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for decision records; distinguish required behavior from illustrative implementation detail.
- Keep scope changes behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Change", "ChangeControlAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.13 and decision records.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.13 Change control and governance

Ownership and lifetime: scope changes

This page defines the ownership and lifetime for Change control and governance, with particular attention to scope changes. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Scope changes is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with review gates is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for scope changes; distinguish required behavior from illustrative implementation detail.
- Keep review gates behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ChangeControlAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ChangeControlAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ChangeControlAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.13 and scope changes.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.13 Change control and governance

Threading model: review gates

This page defines the threading model for Change control and governance, with particular attention to review gates. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Review gates is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with deprecations is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for review gates; distinguish required behavior from illustrative implementation detail.
- Keep deprecations behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Change", "ChangeControlAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.13 and review gates.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.13 Change control and governance

Memory strategy: deprecations

This page defines the memory strategy for Change control and governance, with particular attention to deprecations. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Deprecations is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with decision records is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for deprecations; distinguish required behavior from illustrative implementation detail.
- Keep decision records behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ChangeControlAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ChangeControlAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ChangeControlAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.13 and deprecations.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.13 Change control and governance

Failure handling: decision records

This page defines the failure handling for Change control and governance, with particular attention to decision records. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Decision records is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with scope changes is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for decision records; distinguish required behavior from illustrative implementation detail.
- Keep scope changes behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Change", "ChangeControlAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.13 and decision records.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.13 Change control and governance

Diagnostics: scope changes

This page defines the diagnostics for Change control and governance, with particular attention to scope changes. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Scope changes is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with review gates is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for scope changes; distinguish required behavior from illustrative implementation detail.
- Keep review gates behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ChangeControlAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ChangeControlAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ChangeControlAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.13 and scope changes.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.14 Release readiness and acceptance

Purpose and boundary: release checklist

This page defines the purpose and boundary for Release readiness and acceptance, with particular attention to release checklist. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Release checklist is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with known issues is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for release checklist; distinguish required behavior from illustrative implementation detail.
- Keep known issues behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ReleaseReadinessAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ReleaseReadinessAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ReleaseReadinessAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.14 and release checklist.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.14 Release readiness and acceptance

Architecture: known issues

This page defines the architecture for Release readiness and acceptance, with particular attention to known issues. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Known issues is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with rollback is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for known issues; distinguish required behavior from illustrative implementation detail.
- Keep rollback behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Release", "ReleaseReadinessAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.14 and known issues.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.14 Release readiness and acceptance

Data model: rollback

This page defines the data model for Release readiness and acceptance, with particular attention to rollback. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Rollback is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with support bundle is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for rollback; distinguish required behavior from illustrative implementation detail.
- Keep support bundle behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ReleaseReadinessAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ReleaseReadinessAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ReleaseReadinessAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.14 and rollback.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.14 Release readiness and acceptance

Public API: support bundle

This page defines the public api for Release readiness and acceptance, with particular attention to support bundle. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Support bundle is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with release checklist is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for support bundle; distinguish required behavior from illustrative implementation detail.
- Keep release checklist behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Release", "ReleaseReadinessAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.14 and support bundle.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.14 Release readiness and acceptance

Ownership and lifetime: release checklist

This page defines the ownership and lifetime for Release readiness and acceptance, with particular attention to release checklist. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Release checklist is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with known issues is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for release checklist; distinguish required behavior from illustrative implementation detail.
- Keep known issues behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ReleaseReadinessAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ReleaseReadinessAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ReleaseReadinessAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.14 and release checklist.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.14 Release readiness and acceptance

Threading model: known issues

This page defines the threading model for Release readiness and acceptance, with particular attention to known issues. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Known issues is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with rollback is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for known issues; distinguish required behavior from illustrative implementation detail.
- Keep rollback behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Release", "ReleaseReadinessAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.14 and known issues.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.14 Release readiness and acceptance

Memory strategy: rollback

This page defines the memory strategy for Release readiness and acceptance, with particular attention to rollback. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Rollback is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with support bundle is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for rollback; distinguish required behavior from illustrative implementation detail.
- Keep support bundle behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ReleaseReadinessAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ReleaseReadinessAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ReleaseReadinessAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.14 and rollback.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.14 Release readiness and acceptance

Failure handling: support bundle

This page defines the failure handling for Release readiness and acceptance, with particular attention to support bundle. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Support bundle is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with release checklist is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for support bundle; distinguish required behavior from illustrative implementation detail.
- Keep release checklist behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Release", "ReleaseReadinessAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.14 and support bundle.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.14 Release readiness and acceptance

Diagnostics: release checklist

This page defines the diagnostics for Release readiness and acceptance, with particular attention to release checklist. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Release checklist is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with known issues is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for release checklist; distinguish required behavior from illustrative implementation detail.
- Keep known issues behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ReleaseReadinessAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ReleaseReadinessAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ReleaseReadinessAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.14 and release checklist.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.14 Release readiness and acceptance

Implementation sequence: known issues

This page defines the implementation sequence for Release readiness and acceptance, with particular attention to known issues. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Known issues is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with rollback is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for known issues; distinguish required behavior from illustrative implementation detail.
- Keep rollback behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Release", "ReleaseReadinessAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 12.14 and known issues.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

12.14 Release readiness and acceptance

Verification: rollback

This page defines the verification for Release readiness and acceptance, with particular attention to rollback. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Rollback is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with support bundle is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for rollback; distinguish required behavior from illustrative implementation detail.
- Keep support bundle behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ReleaseReadinessAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ReleaseReadinessAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ReleaseReadinessAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 12.14 and rollback.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.